

PROJET TAXI-DRIVER

SABRI HAMMI



INTRODUCTION

Le projet Taxi Driver demande de résoudre l'environnement Taxi-v3 (Gym) avec un algorithme de reinforcement learning model-free épisodique.

Un taxi doit récupérer un passager et le déposer à la bonne destination dans une grille 5x5 (500 états, 6 actions). Le choix de l'algorithme est libre (Q-Learning, Deep Q-Learning, Monte Carlo...)

Le programme doit proposer un mode utilisateur (réglage des hyperparamètres) et un mode time-limited (paramètres optimisés), afficher les métriques (reward moyen, temps moyen), et inclure un brute-force comme baseline de comparaison.

1. Technologies utilisées

La stack technologique du projet est la suivante :

Python 3.11+ : Langage principal du projet. Choix naturel pour le reinforcement learning grâce à son écosystème de bibliothèques scientifiques et RL. La version 3.11+ apporte des gains de performance significatifs sur l'exécution.

Gymnasium 1.3.0 : Fournit l'environnement Taxi-v3 prêt à l'emploi avec la gestion des états, actions, récompenses et le rendu graphique de la grille. C'est le successeur officiel d'OpenAI Gym, maintenu par la Farama Foundation.

NumPy 2.4.6 : Utilisé pour la création et la manipulation de la Q-table (matrice 500×6), le calcul vectoriel des récompenses cumulées, et le traitement statistique des métriques de benchmark (moyennes, écarts-types).

Matplotlib 3.10.9 : Sert à la visualisation en temps réel pendant l'entraînement et le test (affichage de la grille, courbes de convergence), ainsi qu'à la génération des graphiques de benchmark comparatifs entre les différents algorithmes.

PyTorch 2.12.0 : Utilisé exclusivement pour l'implémentation du Deep Q-Learning (DQN). Il permet de définir l'architecture du réseau de neurones, de gérer la backpropagation et l'optimisation des poids, ainsi que le replay buffer nécessaire à la stabilisation de l'entraînement.

2. ARCHITECTURE DU PROJET

Le projet est organisé en modules séparés, chacun contenant une implémentation indépendante d'un algorithme de résolution. La différenciation de Pulsec se joue sur trois axes cumulatifs :

- **Q_learning/** : contient l'implémentation du Q-Learning tabulaire, algorithme off-policy. Une Q-table de dimension 500×6 est mise à jour par la formule de Bellman à chaque step. C'est la base du projet et la première baseline non-optimisée exigée.
- **Sarsa/** : pour l'implémentation de SARSA, variante on-policy du Q-Learning. La différence principale est que la mise à jour de la Q-table utilise l'action réellement prise au step suivant (et non le max), ce qui donne un comportement plus conservateur. Cela permet une comparaison directe **on-policy** vs **off-policy**.
- **MonteCarlo/** : Dossier d'implémentation de l'algorithme Monte Carlo, une approche épisodique pure recommandée par le sujet. Contrairement aux méthodes TD (Q-Learning, SARSA) qui mettent à jour à chaque step, Monte Carlo attend la fin complète de l'épisode avant de mettre à jour les valeurs estimées.
- **deep_Q_learning/** : implémentation du Deep Q-Learning (DQN). Les valeurs Q sont approximées par un réseau de neurones (PyTorch) au lieu d'une table. L'entraînement utilise un replay buffer et une Huber loss pour stabiliser la convergence. Le modèle est sérialisé automatiquement pour pouvoir être rechargé sans ré-entraînement.

Un algorithme **brute-force** (actions aléatoires) est également mis en place comme baseline de référence (~350 steps contre ~13 pour le Q-Learning optimisé).

2. Mise en place de l'agent Q-Learning

Le projet s'exécute dans un environnement virtuel Python (venv) pour isoler les dépendances du système. Le venv est créé à la racine du projet et contient **Gymnasium 1.3.0**, **NumPy 2.4.6**, **Matplotlib 3.10.9** et **PyTorch 2.12.0**. Cette isolation garantit la reproductibilité, n'importe quel collaborateur peut recréer l'environnement identique via la commande "`pip install -r requirements.txt`".

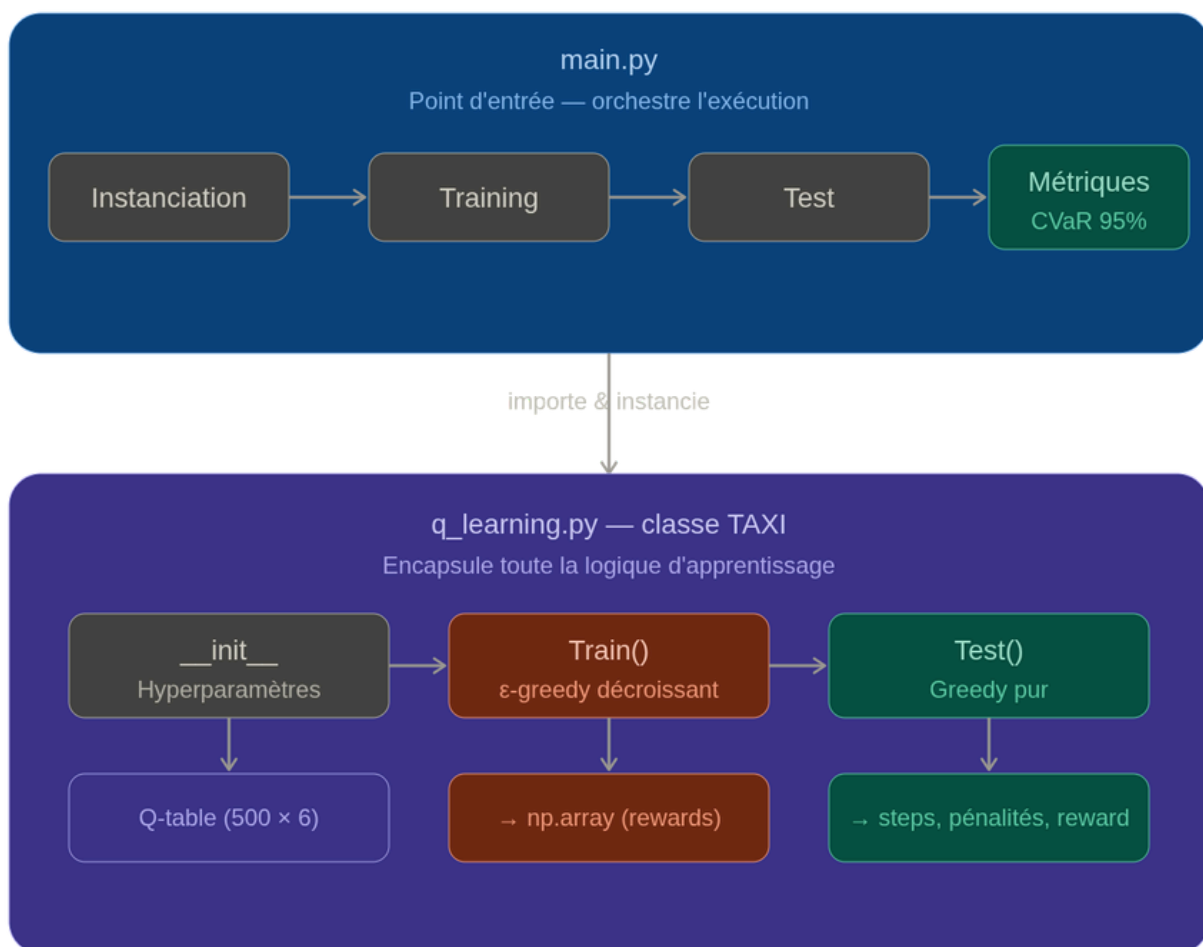
Deux fichiers composent le module Q-Learning :

q_learning.py encapsule toute la logique d'apprentissage dans une classe nommée "TAXI"

La méthode **Train()** exécute l'entraînement avec **epsilon-greedy** décroissant et retourne les rewards par épisode sous forme de `np.array`.

La méthode `test()` évalue l'agent en mode greedy pur et retourne les métriques moyennes (**steps**, **pénalités**, **reward**). Ce découpage en classe rend le module importable, réutilisable et permet d'instancier plusieurs agents avec des hyperparamètres différents pour le benchmark.

`main.py` orchestre l'exécution son rôle est d'instancier l'agent, il lance le training, mesure le temps d'exécution, déclenche la phase de test, puis calcule les métriques statistiques (CVaR à 95% de niveau de confiance). C'est le point d'entrée unique du programme, conforme à l'exigence du sujet de permettre à l'utilisateur de paramétrer le nombre d'épisodes au lancement.



2.1 Résultats du premier entraînement (10k)

```

• (.venv) s4br0nx in ~/TAXI-DRIVER/Q_learning on main λ python3 main.py
Episode    0/10000 | ε=0.9995 | Avg reward (last 1k): -704.00 | Steps: 200
Episode   1000/10000 | ε=0.6062 | Avg reward (last 1k): -507.81 | Steps: 48
Episode   2000/10000 | ε=0.3676 | Avg reward (last 1k): -98.25 | Steps: 58
Episode   3000/10000 | ε=0.2229 | Avg reward (last 1k): -21.51 | Steps: 21
Episode   4000/10000 | ε=0.1352 | Avg reward (last 1k): -6.54 | Steps: 8
Episode   5000/10000 | ε=0.0820 | Avg reward (last 1k): 0.63 | Steps: 14
Episode   6000/10000 | ε=0.0497 | Avg reward (last 1k): 3.33 | Steps: 15
Episode   7000/10000 | ε=0.0302 | Avg reward (last 1k): 5.36 | Steps: 10
Episode   8000/10000 | ε=0.0183 | Avg reward (last 1k): 6.81 | Steps: 6
Episode   9000/10000 | ε=0.0111 | Avg reward (last 1k): 6.88 | Steps: 17

Training terminé – 10000 épisodes
Mean reward : -60.43
Mean steps  : 33.09
Mean penalties : 5.24
Graphique sauvegardé : training_metrics.png

Résultats après 25 épisodes de test :
Average steps      : 27.84
Average penalties: 0.00
Average reward     : -8.52
Train execution time: 2497.07s

CVaR à 0.95 de niveau de confiance : -712.3532934131737

```

La capture montre la convergence de l'agent au fil de l'entraînement. L'epsilon passe de **0.9995** à **0.01**, traduisant la transition progressive de l'exploration vers l'exploitation. Le reward moyen sur les **1 000** derniers épisodes évolue de **-704** (exploration aléatoire) à **+6.88** (politique quasi-optimale), tandis que le nombre de steps par épisode chute de **200** à une **quinzaine**.

Le bilan global de la première phase de training affiche un mean reward de **-60.43** et **33.09** steps moyens, mais ces chiffres sont tirés vers le bas par les premiers épisodes d'exploration. La métrique pertinente est celle des derniers **1 000 épisodes** (**+6.88** de reward, **~15 steps**)

En phase de test (25 épisodes, greedy pur), l'agent atteint **27.84** steps moyens avec **0 pénalité**, confirmant qu'il ne commet plus aucune action illégale. Le reward moyen de **-8.52** s'explique par le coût cumulé des steps (-1 par action) qui dépasse encore le bonus de livraison (**+20**). Ce résultat constitue la baseline non optimisée, à partir de laquelle les optimisations d'hyperparamètres seront mesurées.

Le CVaR à **95%** de **-712.35** reflète le reward moyen des **5%** pires épisodes du training (les tout premiers), confirmant le coût élevé++ de la phase d'exploration initiale.

3. Implémentation de l'agent brute-force (baseline)

L'agent brute-force sert de point de référence pour évaluer les performances des algorithmes de reinforcement learning. Il est placé à la racine du projet car il est commun à tous les benchmarks.

Son fonctionnement est volontairement naïf, à chaque step, l'agent choisit une action aléatoire parmi les 6 disponibles (**South**, **North**, **East**, **West**, **Pickup**, **Dropoff**), sans aucune connaissance de l'environnement ni apprentissage. Il n'y a pas de Q-table, pas de politique, pas de mémoire.

```
(.venv) s4br0nx in ~/TAXI-DRIVER on main λ python3 bruteforce.py  
=== Brute-force Baseline (Random Agent) ===  
  
Épisodes      : 1000  
Mean steps    : 196.42 (±19.63)  
Mean reward   : -769.59 (±105.26)  
Mean penalties : 63.78  
Success rate  : 0.0%  
Execution time : 1.9s  
(.venv) s4br0nx in ~/TAXI-DRIVER on main λ
```

Sur 1000 épisodes, les résultats sont sans appel : 196 steps moyens, un reward moyen de **-769 (±105,26)**, **63** pénalités moyennes par épisode et un taux de succès de **0%**. L'agent ne parvient jamais à livrer le passager à destination. L'environnement Gymnasium tronque d'ailleurs chaque épisode à 200 steps, sans quoi l'agent tournerait indéfiniment.

Ces résultats constituent le plancher de performance. En comparaison, le Q-Learning atteint **~15 steps** et un reward de **+6.88** après **10 000** épisodes d'entraînement, soit une amélioration d'un facteur 13 sur le nombre de steps et un passage d'un reward largement négatif à un reward positif.

Ce contraste justifie le recours à des méthodes d'apprentissage par renforcement pour résoudre Taxi-v3.

4. Implémentation et résultat de l'agent SARSA

SARSA (State-Action-Reward-State-Action) est un algorithme on-policy : contrairement au Q-Learning qui met à jour la Q-table avec la meilleure action possible au step suivant ($\max Q(s', a')$), SARSA utilise l'action réellement choisie par la politique courante. Cette approche rend l'agent plus conservateur dans son apprentissage.

```
(.venv) s4br0nx in ~/TAXI-DRIVER/Sarsa on main λ python3 main.py
Episode    0/10000 | ε=0.9990 | Avg reward (last 1k): -767.00 | Steps: 200
Episode   1000/10000 | ε=0.3673 | Avg reward (last 1k): -384.42 | Steps: 111
Episode   2000/10000 | ε=0.1351 | Avg reward (last 1k): -48.60 | Steps: 14
Episode   3000/10000 | ε=0.0497 | Avg reward (last 1k): -14.27 | Steps: 13
Episode   4000/10000 | ε=0.0183 | Avg reward (last 1k): -7.05 | Steps: 16
Episode   5000/10000 | ε=0.0100 | Avg reward (last 1k): -5.06 | Steps: 12
Episode   6000/10000 | ε=0.0100 | Avg reward (last 1k): -3.90 | Steps: 11
Episode   7000/10000 | ε=0.0100 | Avg reward (last 1k): -4.72 | Steps: 11
Episode   8000/10000 | ε=0.0100 | Avg reward (last 1k): -0.99 | Steps: 11
Episode   9000/10000 | ε=0.0100 | Avg reward (last 1k): -2.06 | Steps: 6
```

Training terminé – 10000 épisodes

Mean reward : -47.43

Mean steps : 36.18

Mean penalties : 3.38

Graphique sauvegardé : sarsa_training_metrics.png

Résultats après 25 épisodes de test :

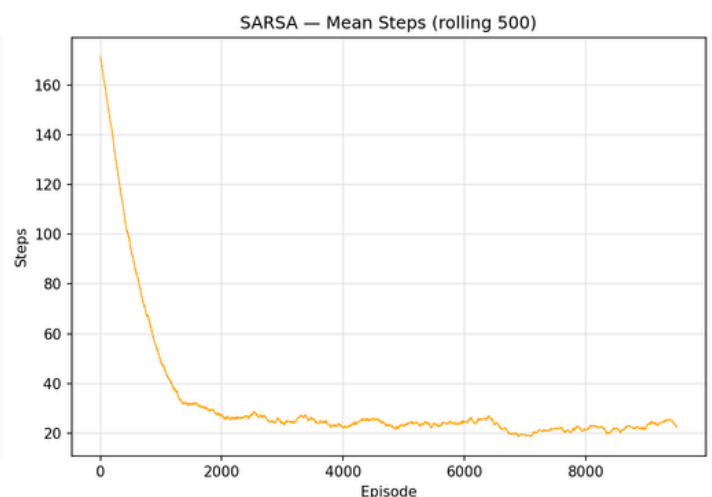
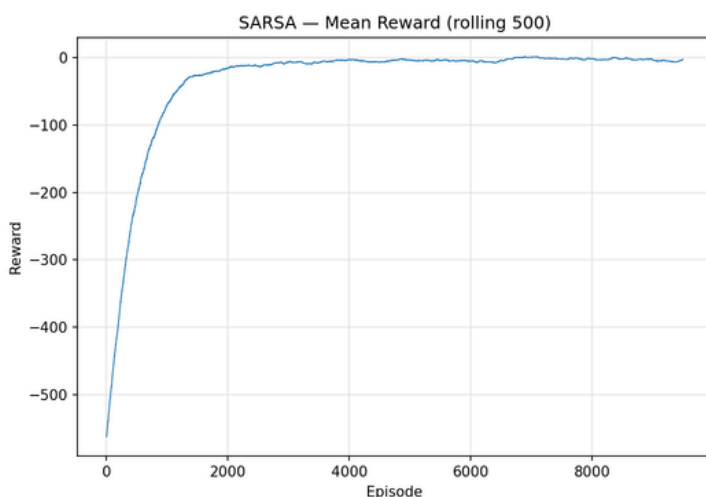
Average steps : 13.72

Average penalties: 0.00

Average reward : 7.28

Train execution time: 27.19s

CVaR à 0.95 de niveau de confiance : -625.2235528942116



L'entraînement sur 10 000 épisodes montre une convergence progressive. L'epsilon décroît de 0.999 à 0.01, et le reward moyen sur les 1 000 derniers épisodes passe de -767 (exploration aléatoire) à -2.06. Le nombre de steps chute de 200 à une dizaine dès l'épisode 3000, signe que l'agent apprend rapidement à naviguer dans la grille. Le bilan global du training affiche un mean reward de -47.43 et 36.18 steps moyens, chiffres tirés vers le bas par la phase d'exploration initiale.

En phase de test (25 épisodes, greedy pur), l'agent atteint 13.72 steps moyens avec un reward de +7.28 et 0 pénalité. L'absence totale d'actions illégales confirme que la politique apprise est propre. Le reward positif signifie que le bonus de livraison (+20) dépasse largement le coût cumulé des déplacements ($\sim 14 \times -1$).

Le temps d'exécution total est de 27.19 secondes, graphique de convergence inclus. Le CVaR à 95% de -625.22 reflète le reward moyen des 5% pires épisodes du training, correspondant aux tout premiers épisodes d'exploration aléatoire.

En comparaison avec le Q-Learning (27.84 steps, reward -8.52 sur 10 000 épisodes), SARSA obtient des performances nettement supérieures avec les mêmes conditions d'entraînement. Cette différence s'explique par l'optimisation des hyperparamètres ($\alpha=0.2$, $\gamma=0.9$, $\text{decay}=0.999$) qui accélère la convergence de la politique on-policy.

5. Implémentation et résultats de l'agent Monte Carlo

Monte Carlo first-visit est un algorithme épisodique pur, il attend la fin complète de chaque épisode pour calculer le retour cumulé réel G et mettre à jour la Q-table. Contrairement au Q-Learning et SARSA qui apprennent à chaque step (bootstrapping), Monte Carlo n'utilise que l'expérience réelle, sans estimation intermédiaire.

L'implémentation utilise un Monte Carlo incrémental avec learning rate ($Q(s,a) += \alpha * [G - Q(s,a)]$) plutôt que la moyenne classique des retours, qui s'est avérée inefficace car elle empêche l'agent de s'adapter aux améliorations récentes de sa politique.


```
(.venv) s4br0nx in ~/TAXI-DRIVER/MonteCarlo on main λ python3 main.py
Episode 47000/50000 |  $\epsilon=0.0100$  | Avg reward (last 1k): -84.68 | Steps: 9
Episode 48000/50000 |  $\epsilon=0.0100$  | Avg reward (last 1k): -76.88 | Steps: 200
Episode 49000/50000 |  $\epsilon=0.0100$  | Avg reward (last 1k): -88.13 | Steps: 10
```

Training terminé – 50000 épisodes

Mean reward : -121.03

Mean steps : 101.96

Mean penalties : 3.40

Graphique sauvegardé : monte_carlo_training_metrics.png

Résultats après 25 épisodes de test :

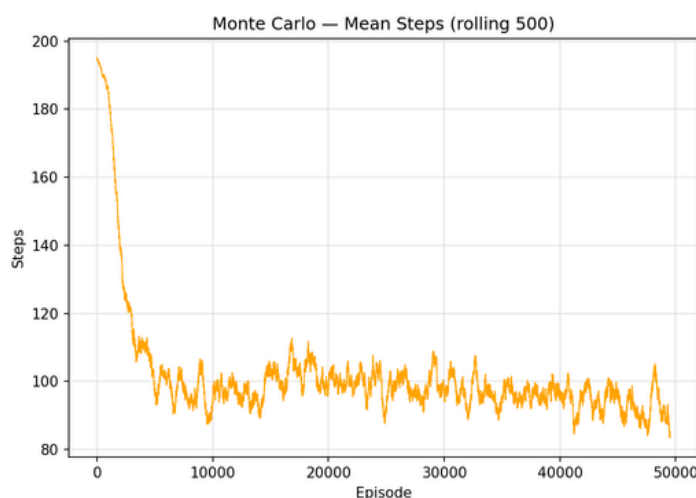
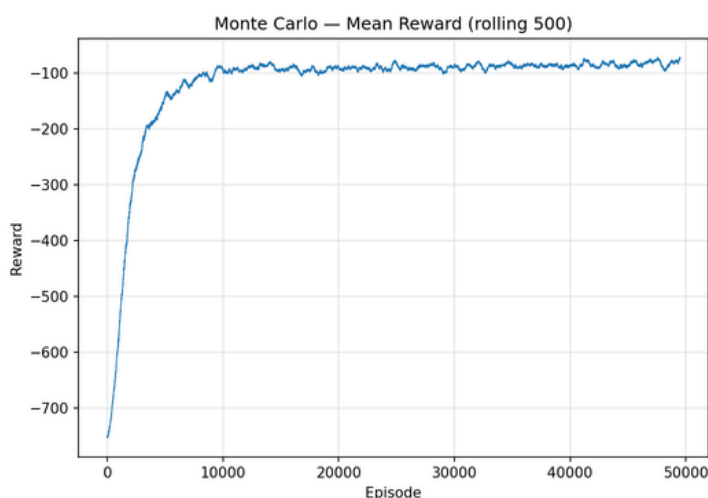
Average steps : 95.60

Average penalties: 0.00

Average reward : -83.84

Train execution time: 53.45s

CVaR à 0.95 de niveau de confiance : -609.7912834866054



L'entraînement sur 50 000 épisodes montre une convergence lente et instable. Le reward moyen sur les derniers 1 000 épisodes oscille entre -76 et -88, sans jamais se stabiliser. Certains épisodes se résolvent en moins de 10 steps tandis que d'autres atteignent la troncature à 200, ce qui traduit une politique encore fragile sur certains états. Le bilan global du training affiche un mean reward de -121.03, 101.96 steps moyens et 3.40 pénalités, fortement impactés par la phase d'exploration initiale.

En phase de test (25 épisodes, greedy pur), l'agent atteint 95.60 steps moyens avec un reward de -83.84 et 0 pénalité. L'absence de pénalités confirme que l'agent a appris à ne pas effectuer d'actions illégales, mais le nombre élevé de steps montre qu'il ne maîtrise pas encore les trajets optimaux. Le reward négatif s'explique par le coût cumulé des déplacements ($\sim 96 \times -1$) qui dépasse largement le bonus de livraison (+20).

Le temps d'exécution est de 53.45 secondes pour 50 000 épisodes, soit 5 fois plus d'épisodes que le Q-Learning et SARSA pour des résultats

nettement inférieurs. Le CVaR à 95% de -609.79 reflète les pires épisodes du training.

Ces performances illustrent la limite intrinsèque de Monte Carlo sur Taxi, les premiers épisodes font systématiquement 200 steps (troncature Gymnasium), produisant des retours cumulés très bruités qui ralentissent la convergence. Les méthodes TD, qui apprennent à chaque step, s'adaptent beaucoup plus vite dans ce contexte. Monte Carlo reste néanmoins pertinent comme point de comparaison pour démontrer l'avantage du bootstrapping sur un environnement à épisodes longs.

6. Implémentation et résultats de l'agent Deep Q-Learning (DQN)

Le Deep Q-Learning remplace la Q-table par un réseau de neurones à deux couches cachées (128 et 64 neurones, activation ReLU) qui approxime la fonction $Q(s, a)$. L'état est encodé en one-hot (500 dimensions) et le réseau produit les 6 Q-values correspondant à chaque action.

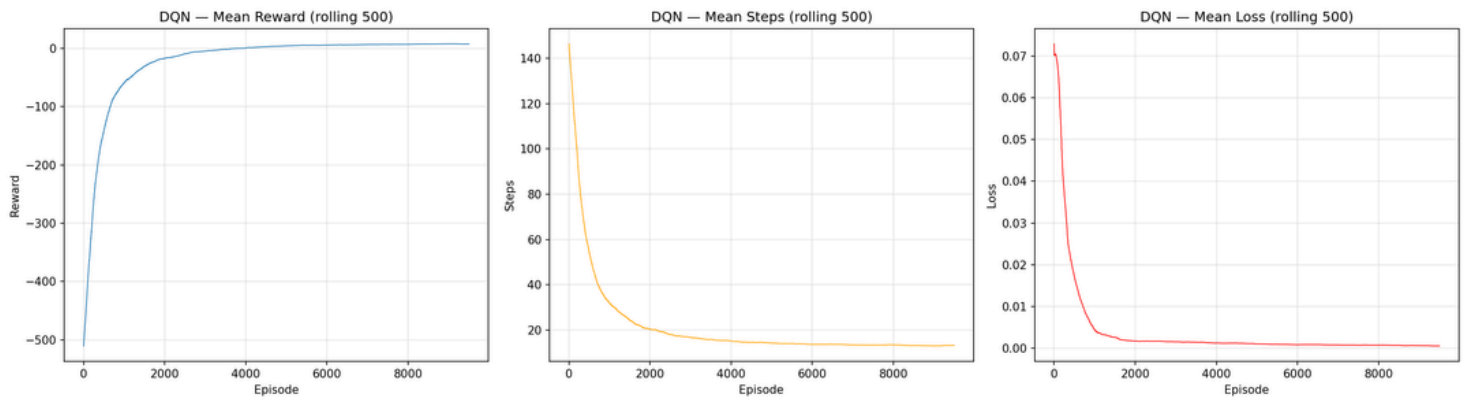
L'entraînement est stabilisé par trois mécanismes, un experience replay de 50 000 transitions, un target network mis à jour tous les 10 épisodes, et un gradient clipping à norme 1.0.

```
(.venv) s4br0nx in ~/TAXI-DRIVER/deep Q learning on main λ python3 main.py
Episode 8000/10000 | ε=0.0183 | Avg reward (last 1k): 6.72 | Steps: 11 | Loss: 0.0008
Episode 9000/10000 | ε=0.0111 | Avg reward (last 1k): 7.12 | Steps: 10 | Loss: 0.0007

Training terminé – 10000 épisodes
  Mean reward : -35.35
  Mean steps  : 24.67
  Mean penalties : 3.47
Graphique sauvegardé : dqn_training_metrics.png

Résultats après 25 épisodes de test :
  Average steps      : 12.56
  Average penalties: 0.00
  Average reward     : 8.44
Modèle sauvegardé : dqn_model.pth
Train execution time: 476.22s

CVaR à 0.95 de niveau de confiance : -539.8602794411178
```



L'entraînement sur 10 000 épisodes montre une convergence rapide et stable. Le reward moyen sur les derniers 1 000 épisodes atteint +6.72 à l'épisode 8 000 puis +7.12 à l'épisode 9 000, avec des steps oscillant entre 10 et 11. La loss descend à 0.0008 puis 0.0007, confirmant que le réseau a appris une approximation précise de la Q-fonction. Le bilan global du training affiche un mean reward de -35.35 et 24.67 steps moyens, tirés vers le bas par la phase d'exploration initiale, avec 3.47 pénalités moyennes concentrées sur les premiers épisodes.

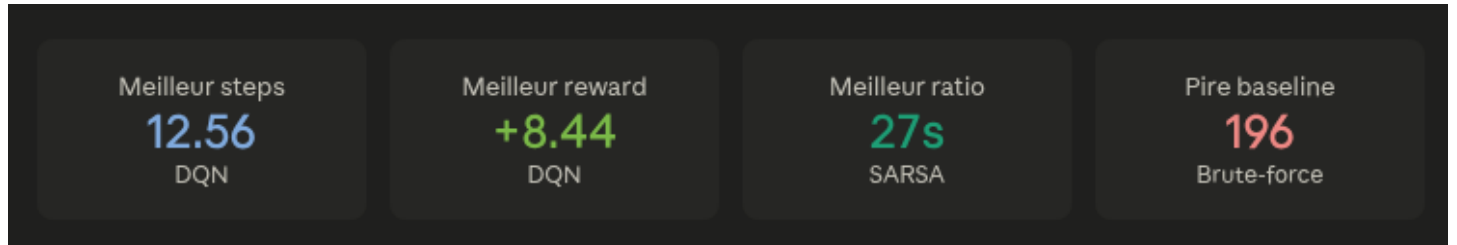
En phase de test (25 épisodes, greedy pur), l'agent atteint 12.56 steps moyens avec un reward de +8.44 et 0 pénalité. Ce sont les meilleurs résultats de tous les algorithmes implémentés lors de ce projet. Le reward positif élevé signifie que le taxi livre le passager en environ 13 step/déplacements, bien en dessous des 20 steps annoncés comme objectif par le sujet. Le modèle entraîné est sérialisé dans `dqn_model.pth`, permettant de tester sans ré-entraînement.

Le temps d'exécution est de 476.22 secondes (~8 minutes) pour 10 000 épisodes, soit 17 fois plus lent que le Q-Learning et SARSA pour le même nombre d'épisodes. Ce surcoût s'explique par le principe de forward pass et la backpropagation à chaque step, ainsi que le sampling du replay buffer. Le CVaR à 95% de -539.86 reflète les pires épisodes du training.

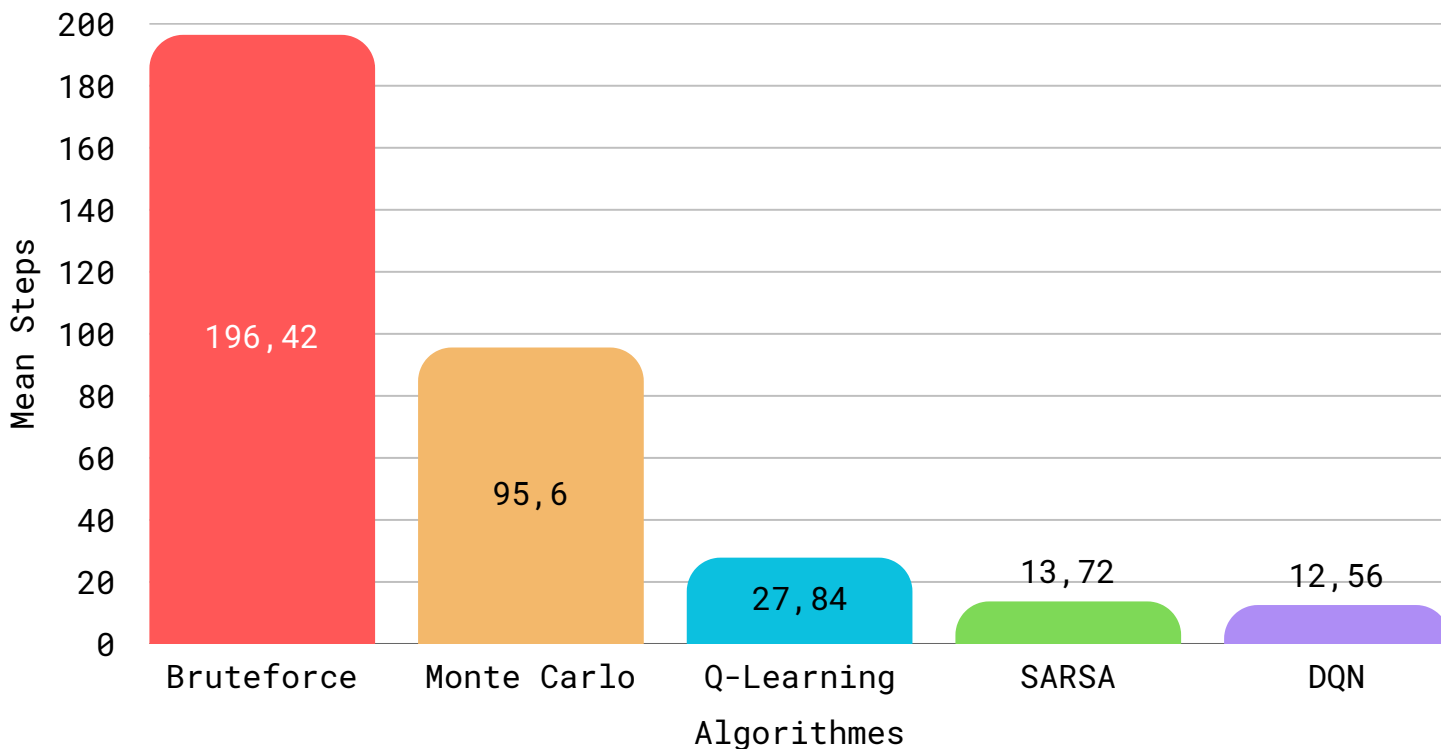
Le DQN illustre le compromis central du reinforcement learning, il obtient la meilleure politique finale mais au prix d'un coût computationnel significativement plus élevé. Sur un environnement à espace d'états discret et modeste comme Taxi-v3 (500 états), la Q-table tabulaire reste plus efficiente en temps. L'avantage du DQN se révèle sur des environnements à espace continu ou de grande dimension, où une table n'est plus viable.

7. Benchmark comparatif

Le comparatif suivant synthétise les performances de chaque algorithme mesuré en phase de test (greedy pur, $\epsilon=0$)

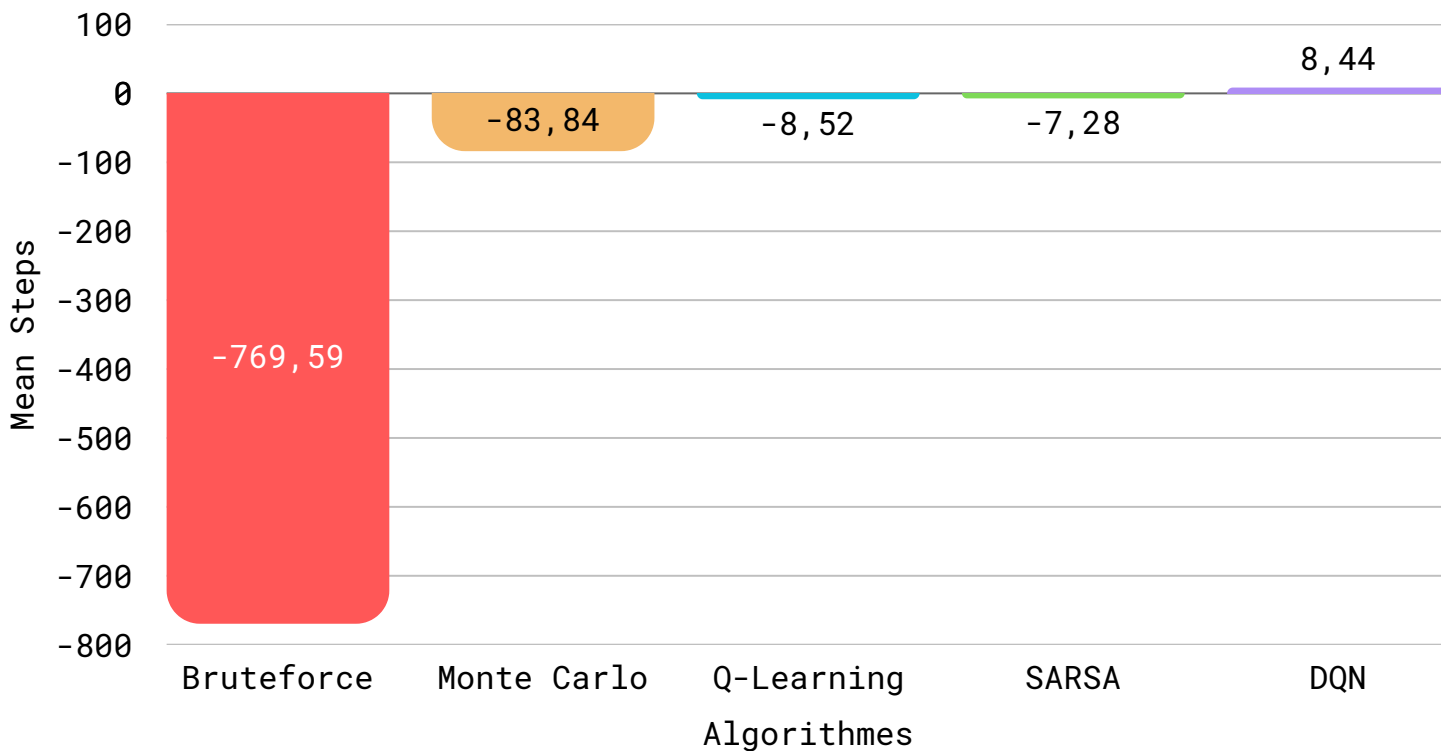


Mean Steps en test (moins = mieux)

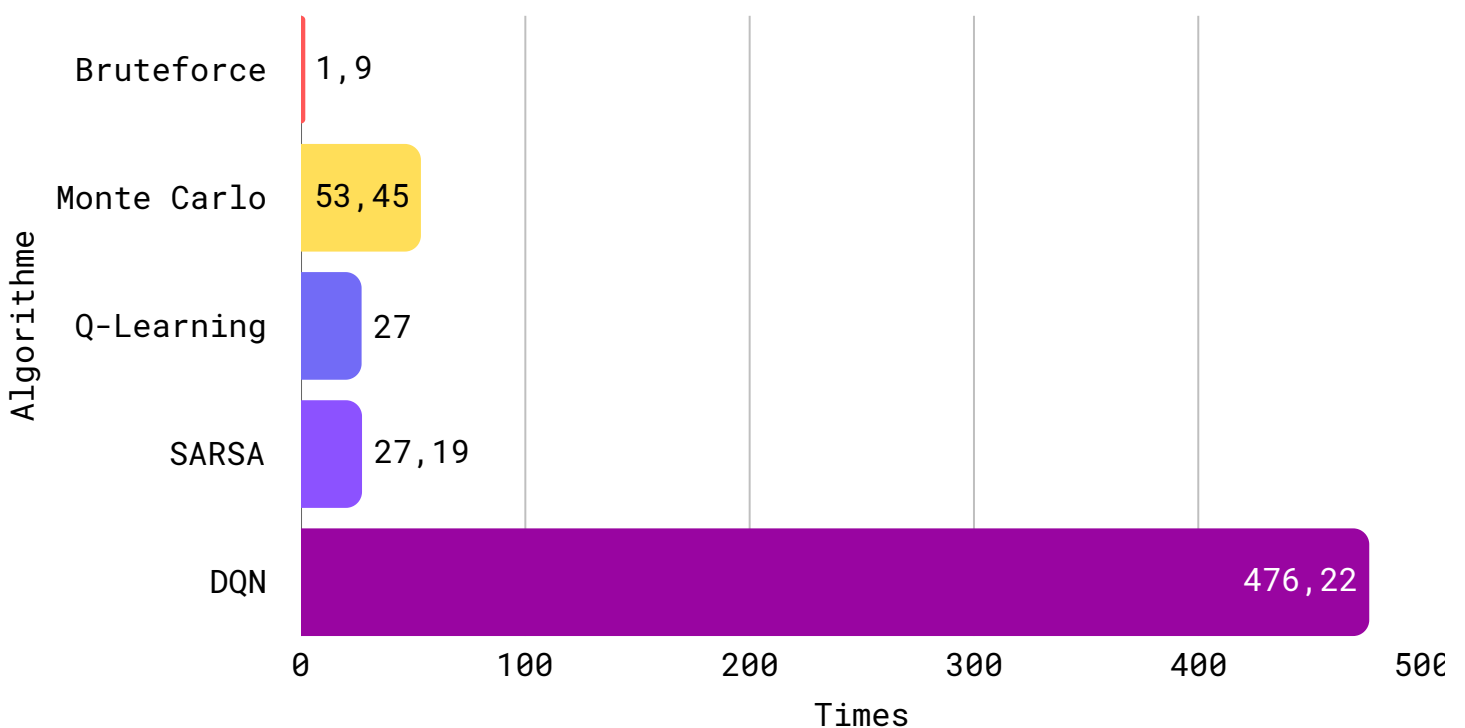


Le Deep Q-Learning est largement devant

Mean Reward en test (plus = mieux)



Temps d'entrainement (secondes)



Trois constats se dégagent de ce benchmark.

Premièrement, tous les algorithmes Reinforcement learning éliminent les pénalités. Même Monte Carlo, malgré ses performances globales limitées, apprend à ne jamais effectuer d'action illégale. Le brute-force accumule 63 pénalités par épisode.

Deuxièmement, les méthodes TD (Q-Learning, SARSA, DQN) surpassent largement Monte Carlo. Le bootstrapping permet de mettre à jour la Q-table à chaque step plutôt qu'en fin d'épisode, exploitant immédiatement chaque interaction avec l'environnement. Sur Taxi-v3 où les épisodes d'exploration durent 200 steps, Monte Carlo accumule des retours cumulés très bruités qui ralentissent la convergence. Il nécessite 50 000 épisodes pour atteindre un plateau à -83 de reward, là où SARSA atteint +7.28 en 10 000 épisodes.

Troisièmement, le DQN obtient la meilleure politique (12.56 steps, +8.44 reward) mais au prix d'un temps d'entraînement 17 fois supérieur à SARSA pour un gain marginal (+1.16 de reward, -1.16 steps). Sur un espace d'états discret de 500, le réseau de neurones est surdimensionné par rapport à une Q-table. Le rapport performance/coût favorise les méthodes tabulaires sur cet environnement.

7.1

Impact des hyperparamètres

L'optimisation des hyperparamètres a un impact déterminant sur les performances, particulièrement visible sur SARSA. Avec les paramètres par défaut du Q-Learning ($\alpha=0.1$, $\gamma=0.6$, $\text{decay}=0.9995$), SARSA stagnait à 128 steps et -120 de reward. Le passage à $\alpha=0.2$, $\gamma=0.9$ et $\text{decay}=0.999$ a divisé les steps par 9 et fait passer le reward de -120 à +7.28. Le learning rate plus élevé accélère l'intégration des nouvelles expériences dans la Q-table, le gamma plus élevé pousse l'agent à valoriser les récompenses futures (la livraison à +20), et le decay plus rapide réduit l'exploration parasite en fin de training.

Pour le DQN, la Huber loss (plutôt que MSE) et le gradient clipping à 1.0 se sont révélés essentiels à la stabilité de l'entraînement. Sans gradient clipping, le réseau diverge fréquemment lors des premiers épisodes où les rewards négatifs importants produisent des gradients explosifs.

7.2

Stratégie d'optimisation des rewards

L'environnement Taxi-v3 utilise un système de récompenses fixe : -1 par step, +20 pour une livraison réussie, -10 pour une action illégale. Ce

système pénalise naturellement les trajectoires longues et récompense la résolution rapide, ce qui aligne directement la fonction de récompense avec l'objectif d'optimisation du nombre de steps.

Le CVaR à 95% diminue progressivement entre les algorithmes : -712.35 (Q-Learning), -625.22 (SARSA), -609.79 (Monte Carlo), -539.86 (DQN). Cette amélioration du worst-case traduit une exploration initiale de plus en plus efficace au fil des implémentations, chaque algorithme bénéficiant des leçons tirées du précédent en termes de tuning des hyperparamètres.

8. Implémentation des modes d'exécution et de l'affichage visuel

Le sujet exige deux modes d'exécution distincts, ainsi que l'affichage visuel d'épisodes de test. Les fichiers `main.py` de chaque algorithme ont été refactorisés pour répondre à ces exigences.

Au lancement, le programme affiche un menu interactif proposant deux modes. Le mode utilisateur permet de régler manuellement chaque hyperparamètre, learning rate (α), discount factor (γ), epsilon initial, epsilon minimum, epsilon decay, et pour le DQN le batch size. Chaque paramètre propose une valeur par défaut entre crochets, appliquée si l'utilisateur appuie sur Entrée sans saisir de valeur. Ce mode permet d'expérimenter librement avec les paramètres pour observer leur impact sur la convergence et les performances, conformément à l'exigence de benchmark du sujet.

Le mode time-limited utilise des paramètres pré-optimisés déterminés lors de la phase de benchmark. L'utilisateur ne saisit que le nombre d'épisodes de training et de test. Les paramètres optimisés sont affichés au lancement pour transparence. Ce mode garantit les meilleures performances sans nécessiter de connaissance préalable des hyperparamètres.

Les deux modes demandent systématiquement le nombre d'épisodes d'entraînement et de test, conformément à l'exigence du sujet que ces valeurs soient saisies par l'utilisateur au lancement du programme. Chaque mode propose également d'afficher les épisodes de test visuellement. Lorsque l'affichage est activé, le programme rend la grille Taxi-v3 via matplotlib à chaque step de chaque épisode de test, avec un délai de 0.3 seconde entre les actions et une pause d'une seconde à la fin de chaque épisode. L'utilisateur observe ainsi l'agent naviguer dans la grille, récupérer le passager et le déposer à destination. Lorsque l'affichage est

désactivé, seules les métriques finales sont affichées, ce qui accélère significativement l'exécution.

9. Conclusion

Le projet démontre la progression du brute-force (196 steps, -769 reward) au DQN (12.56 steps, +8.44 reward), soit une amélioration d'un facteur 15 sur le nombre de steps et un passage d'un reward catastrophique à un reward positif. L'objectif du sujet (passer de ~350 steps à ~20) est largement dépassé avec un agent capable de livrer le passager en 13 steps en moyenne.

Le meilleur rapport performance/coût revient à SARSA (13.72 steps, +7.28 reward, 27 secondes d'entraînement) : il atteint 97% de la performance du DQN en 17 fois moins de temps. Le DQN reste pertinent comme démonstration de l'approximation de fonction, mais son surcoût computationnel ne se justifie pas sur un espace d'états aussi modeste que Taxi-v3.

Monte Carlo, bien que recommandé par le sujet, s'avère le moins adapté à cet environnement en raison de sa dépendance aux épisodes complets et de la variance élevée des retours cumulés sur des épisodes longs. Son inclusion dans le benchmark reste néanmoins précieuse pour illustrer le compromis biais-variance et justifier le recours aux méthodes TD.